

Performance Engineering (2360013) — Assignment 1

Rami Habib, ID 325420180. Instructor: Nadav Amit.

VM: course-08, a KVM guest with an Intel Xeon Gold 5420+, 4 vCPUs, kernel 7.0.0-15, ext4.

Short version of my findings:

- **Task 1:** the pipe gets faster under CPU load because the scheduler co-locates the two pipe processes on one core. With idle CPUs around it spreads them across cores, and every round-trip then pays a cross-core wakeup; load removes the idle CPUs so they end up together.
- **Task 2:** v2 reads faster because its 4 MiB writes make the kernel store the file in 2 MiB folios, which get mapped as huge pages. That cuts TLB misses by about 475x on a workload that is otherwise TLB-bound.

The raw command outputs I relied on are in the appendix.

Task 1: Pipe latency under background load

The phenomenon

`lat_pipe` measures the round-trip latency of a Unix pipe. On its own it reports about 11 μ s. When I run `stress-ng --cpu 3` in the background, it drops to about 5 μ s, so adding CPU load makes it roughly 2x faster, which is the opposite of what I expected.

Baseline, 10 runs each, median:

	latency
no load	11.1 μ s
with load (stress-ng --cpu 3)	5.3 μ s

The effect is much bigger than the run-to-run spread, so it is real.

What could cause it

The benchmark does a fixed amount of work per round-trip, so the latency is basically (work / CPU speed) + (cost of waking up the other process). Background load could help in three ways: the CPU could run at a higher clock under load (frequency), an idle CPU could be paying a sleep/wakeup cost (idle states), or the two pipe processes could be placed differently by the scheduler (placement).

This is a KVM guest with no `cpufreq` or `cpuidle` drivers exposed (I checked with `cpupower`), so I could not toggle frequency or C-states directly. I probed indirectly with `perf` instead.

Ruling out frequency

I compared `cycles` to `ref-cycles` in `perf stat`. `ref-cycles` count at the fixed nominal clock, so their ratio is the real frequency. It was the same (1.35) with and without load, which means that when the CPU is actually running it runs at the same speed either way. Frequency is not it.

Separating idle from placement

Pinning both pipe processes to a single core (no load) gave 5.4 μ s, as fast as the loaded case. But that changes two things at once (the core can no longer go idle, and both ends are now on the same core), so it does not tell me which one matters.

To separate them I pinned both ends to two cores and ran it twice: once letting the cores idle, once with a busy spinner on each core so they never idle.

	latency
same core	5.4 μ s
two cores, idle	11.2 μ s
two cores, kept busy	10.9 μ s

Keeping the cores busy did not help, so idle/halt cost is not the cause. The thing that matters is same-core versus cross-core.

The cause: scheduler placement

When there are idle CPUs, the scheduler spreads the two pipe processes onto different cores, so every round-trip is a cross-core wakeup: the kernel has to wake the other core and the pipe data has to move between the two cores' caches. Background load keeps all the CPUs busy, so the scheduler puts both processes on the same core, where a context switch is about twice as cheap.

Two pieces of evidence back this up. A perf sched callgraph of the no-load run shows the two pipe processes on different cores (cpu0 and cpu3), each handing the CPU to the idle task when it blocks. And an off-CPU latency histogram shows the per-handoff wait dropping from the 8-16 μ s bucket (no load) to the 2-4 μ s bucket (with load), which matches the latency change.

Kernel source

The placement decision is in `kernel/sched/fair.c`. On a wakeup, `select_task_rq_fair` takes the fast path to `select_idle_sibling`, and inside it (line 110) `select_idle_cpu` scans the cache domain for an idle CPU and returns it. So with idle CPUs available the woken pipe process is sent to a free core, away from its partner. Under load there is no idle CPU to find, so it stays on the waker's core and the two processes run together. That co-location is why load makes the pipe faster.

Task 2: Two prepares, one run

The phenomenon

I create the same 64 MiB file twice: v1 with the default (~16 KiB) write block size, and v2 with 4 MiB writes. Then I run the same random 4 KiB mmap read test against each. v2 comes out about 9% faster:

	reads/s
v1	1,942,670
v2	2,124,442 (+9.4%)

Both files hold the same data and the read command is identical, so the only difference is how each file was written during prepare.

How I approached it

The difference has to come from how each file was written, so I considered three possible causes and tested each: disk fragmentation, CPU cache (LLC) behaviour, and how the kernel stores the file in the page cache. I ruled out the first two with counters and traced the third to the mechanism.

Ruling out disk fragmentation

The obvious guess is that v1's small writes fragment the file on disk. But the file is only 64 MiB and the VM has plenty of RAM, so after prepare the whole file sits in the page cache. perf stat confirms this: during the run the page faults are all minor (page-faults equals minor-faults, zero major faults), which means nothing is read from disk. If the disk is never touched, its layout cannot affect the result. filefrag also shows both files are basically contiguous (2 extents vs 1). So fragmentation is not the cause.

Ruling out the CPU cache

The next guess is that v2 has better last-level-cache behaviour. If so it would show fewer cache-misses. perf stat shows the opposite: cache-misses are about equal, 130.6M for v1 and 126.8M for v2, at similar cache-references. So the speedup is not a cache effect, which is what later points me at the TLB instead.

What actually differs: folio size

The real difference is how the kernel stores each file in the page cache. I traced the folio sizes built during prepare with bpftrace (on `mm_filemap_add_to_page_cache`):

	folios in the page cache
v1 (16 KiB writes)	16 KiB folios
v2 (4 MiB writes)	2 MiB folios (the whole file)

Bigger writes let the kernel build bigger folios.

Why bigger folios read faster

v2's 2 MiB folios get mapped as huge pages. `/proc/<pid>/smaps_rollup` during a run shows:

	FilePmdMapped
v1	0
v2	65,536 kB (the whole 64 MiB file)

This matters because the benchmark is limited by the TLB. With 4 KiB pages the TLB only covers a few MiB, so random reads across a 64 MiB file miss almost every time. `perf stat` shows exactly that:

	dTLB-load-misses
v1	8,790,240
v2	18,488 (about 475x fewer)

A handful of 2 MiB pages cover the whole file, so v2 almost never misses the TLB. Fewer TLB misses means fewer page-table walks, and that is the ~9% speedup.

Checking the size of the effect independently

To confirm that 2 MiB mapping really is worth this much on this CPU, I ran the same random 4 KiB read pattern over a mapping backed by explicit 2 MiB pages (MAP_HUGETLB) versus normal 4 KiB pages. The dTLB misses dropped from about 195M to a few thousand, and throughput rose +12.7% on a 256 MiB working set and +20.2% on 1 GiB. That is the same order as the file gap, and it isolates the effect to the page size alone, so the mechanism is not specific to the file path.

Kernel source

- `mm/readahead.c`, `page_cache_ra_order`: the folio order is capped by the I/O size (`new_order = min(..., ilog2(ra->size))`). v2's 4 MiB writes allow a 2 MiB folio; v1's small writes do not.
- `mm/memory.c`, `do_set_pmd`: installs a 2 MiB huge-page mapping, but only when the folio is already PMD-sized. It fires for v2 and falls back to 4 KiB pages for v1.

Summary

prepare block size (4 MiB vs 16 KiB) sets the page-cache folio order (2 MiB vs 16 KiB). v2's 2 MiB folios are mapped as huge pages, which cuts the dTLB misses that dominate this random-read workload (8.79M down to 18K), and that is the ~9% speedup. It is a TLB effect, not a cache effect, and not a disk effect.

Appendix: raw command outputs

Task 1

logs/task1_20260610_1427.log

baseline: lat_pipe x10, no-load then with-load

```
Script started on 2026-06-10 14:27:48+00:00 [COMMAND="bash /tmp/task1_p01.sh" <not executed on terminal>]
##### PHASE 0 - environment as shipped #####
wrote task1/env_asshipped.txt
##### PHASE 1 - baseline no-load (n=10) #####
# Task1 Phase 1 - lat_pipe, NO load, n=10 - 2026-06-10T14:27:48+00:00
Pipe latency: 11.2042 microseconds
Pipe latency: 10.4985 microseconds
Pipe latency: 11.1315 microseconds
Pipe latency: 11.1770 microseconds
Pipe latency: 11.1627 microseconds
Pipe latency: 11.1287 microseconds
Pipe latency: 11.1157 microseconds
Pipe latency: 11.0622 microseconds
Pipe latency: 10.5657 microseconds
Pipe latency: 9.8751 microseconds
##### PHASE 1 - baseline with load: stress-ng --cpu 3 (n=10) #####
# Task1 Phase 1 - lat_pipe, WITH load (stress-ng --cpu 3), n=10 - 2026-06-10T14:28:04+00:00
Pipe latency: 5.2355 microseconds
Pipe latency: 5.6613 microseconds
Pipe latency: 5.2755 microseconds
Pipe latency: 6.3949 microseconds
Pipe latency: 5.5291 microseconds
Pipe latency: 5.5314 microseconds
Pipe latency: 4.7368 microseconds
Pipe latency: 4.9441 microseconds
Pipe latency: 5.0993 microseconds
Pipe latency: 4.8185 microseconds
##### DONE #####

Script done on 2026-06-10 14:30:34+00:00 [COMMAND_EXIT_CODE="0"]
```

task1/p2_perfstat_noload.txt

perf stat, no load (frequency check)

Pipe latency: 69.6173 microseconds

Performance counter stats for '/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe':

1513.48 msec	task-clock
1619968404	cycles
667048034	instructions
1202635280	ref-cycles
29252	context-switches
2	cpu-migrations

1.571171194 seconds time elapsed

0.587697000 seconds user

0.891950000 seconds sys

task1/p2_perfstat_load.txt

perf stat, with load

Pipe latency: 69.1056 microseconds

Performance counter stats for '/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe':

28749.27 msec	task-clock
73484767995	cycles
26922915348	instructions
54567480720	ref-cycles
33982	context-switches
6	cpu-migrations

29.746661716 seconds time elapsed

27.719841000 seconds user

1.001958000 seconds sys

task1/p2_pin1core_noload.txt

both pipe ends pinned to one core

Pipe latency: 5.4472 microseconds
Pipe latency: 5.3650 microseconds
Pipe latency: 5.4265 microseconds
Pipe latency: 5.4146 microseconds
Pipe latency: 5.4262 microseconds

task1/p2b_pin2core_idle.txt

two cores, allowed to idle

Pipe latency: 11.2613 microseconds
Pipe latency: 11.3220 microseconds
Pipe latency: 11.2336 microseconds
Pipe latency: 11.2479 microseconds
Pipe latency: 11.0964 microseconds

task1/p2b_pin2core_busy.txt

two cores, kept busy

```
Pipe latency: 10.3362 microseconds
Pipe latency: 10.9540 microseconds
Pipe latency: 11.0637 microseconds
Pipe latency: 11.0859 microseconds
Pipe latency: 11.1015 microseconds
```

task1/p4_offcpu_hist_noload.txt

off-CPU latency histogram, no load

```
stdin:2:125-151: WARNING: Return value discarded.
tracepoint:sched:sched_switch /args->next_comm=="lat_pipe"/ { $s=@t[args->next_pid]; if($s){
@off_us=hist((nsecs-$s)/1000); delete(@t[args->next_pid]); } }
```

~~~~~

Attached 2 probes

Pipe latency: 17.5059 microseconds

@off\_us:

|              |        |  |  |
|--------------|--------|--|--|
| [2, 4)       | 9993   |  |  |
| [4, 8)       | 353    |  |  |
| [8, 16)      | 139094 |  |  |
| [16, 32)     | 33289  |  |  |
| [32, 64)     | 944    |  |  |
| [64, 128)    | 157    |  |  |
| [128, 256)   | 36     |  |  |
| [256, 512)   | 16     |  |  |
| [512, 1K)    | 17     |  |  |
| [1K, 2K)     | 5      |  |  |
| [2K, 4K)     | 2      |  |  |
| [4K, 8K)     | 0      |  |  |
| [8K, 16K)    | 0      |  |  |
| [16K, 32K)   | 0      |  |  |
| [32K, 64K)   | 0      |  |  |
| [64K, 128K)  | 0      |  |  |
| [128K, 256K) | 0      |  |  |
| [256K, 512K) | 0      |  |  |
| [512K, 1M)   | 2      |  |  |

@t[77549]: 1643768205293775

@t[77548]: 1643768206232093

@t[77535]: 1643768206467861

## **task1/p4\_offcpu\_hist\_load.txt**

*off-CPU latency histogram, with load*



```
stdin:2:125-151: WARNING: Return value discarded.
tracepoint:sched:sched_switch /args->next_comm=="lat_pipe"/ { $s=@t[args->next_pid]; if($s){
@off_us=hist((nsecs-$s)/1000); delete(@t[args->next_pid]); } }
```

~~~~~

Attached 2 probes

Pipe latency: 6.4716 microseconds

@off_us:

[2, 4)	311579		@@	
[4, 8)	40707		@@@@@	
[8, 16)	5454			
[16, 32)	279			
[32, 64)	89			
[64, 128)	59			
[128, 256)	65			
[256, 512)	45			
[512, 1K)	18			
[1K, 2K)	14			
[2K, 4K)	21			
[4K, 8K)	0			
[8K, 16K)	0			
[16K, 32K)	0			
[32K, 64K)	0			
[64K, 128K)	0			
[128K, 256K)	0			
[256K, 512K)	1			
[512K, 1M)	2			

@t[77602]: 1643782344750226

@t[77601]: 1643782344905247

@t[77593]: 1643782345054053

task1/p4_callgraph_noload.txt

perf sched callgraph, no load

```

# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 682K of event 'sched:sched_switch'
# Event count (approx.): 682111
#
# Children      Self  Trace output
# .....
#
16.63%    16.63% lat_pipe:77440 [120] S ==> swapper/0:0 [120]
|
---0x60deb1a649e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main
0x60deb1a6496b
0x60deb1a693f4
0x60deb1a68f89
0x60deb1a64d26
0x60deb1a64be8
__libc_read (inlined)
__syscall_cancel
__internal_syscall_cancel (inlined)
entry_SYSCALL_64_after_hwframe
do_syscall_64
x64_sys_call
__x64_sys_read
ksys_read
vfs_read
anon_pipe_read
schedule
__schedule
perf_trace_sched_switch

14.74%    14.74% lat_pipe:77439 [120] S ==> swapper/3:0 [120]
|
---0x60deb1a649e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main
0x60deb1a6496b
0x60deb1a693f4
0x60deb1a68fd4
0x60deb1a64b10
__libc_read (inlined)
__syscall_cancel
__internal_syscall_cancel (inlined)
entry_SYSCALL_64_after_hwframe
do_syscall_64
x64_sys_call
__x64_sys_read
ksys_read
vfs_read
anon_pipe_read
schedule
__schedule
perf_trace_sched_switch

12.93%    12.93% lat_pipe:77442 [120] S ==> swapper/0:0 [120]
|
---0x59949e3f59e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main

```

```
0x59949e3f596b
0x59949e3fa3f4
|
--12.93%--0x59949e3f9fd4
    0x59949e3f5b10
    __libc_read (inlined)
    __syscall_cancel
    __internal_syscall_cancel (inlined)
    entry_SYSCALL_64_after_hwframe
    do_syscall_64
    x64_sys_call
    __x64_sys_read
    ksys_read
    vfs_read
```

task1/p4_select_task_rq_fair.txt

kernel source: select_task_rq_fair (mm/.../sched/fair.c)

```

static int
select_task_rq_fair(struct task_struct *p, int prev_cpu, int wake_flags)
{
    int sync = (wake_flags & WF_SYNC) && !(current->flags & PF_EXITING);
    struct sched_domain *tmp, *sd = NULL;
    int cpu = smp_processor_id();
    int new_cpu = prev_cpu;
    int want_affine = 0;
    /* SD_flags and WF_flags share the first nibble */
    int sd_flag = wake_flags & 0xF;

    /*
     * required for stable ->cpus_allowed
     */
    lockdep_assert_held(&p->pi_lock);
    if (wake_flags & WF_TTWU) {
        record_wakee(p);

        if ((wake_flags & WF_CURRENT_CPU) &&
            cpumask_test_cpu(cpu, p->cpus_ptr))
            return cpu;

        if (!is_rd_overutilized(this_rq()->rd)) {
            new_cpu = find_energy_efficient_cpu(p, prev_cpu);
            if (new_cpu >= 0)
                return new_cpu;
            new_cpu = prev_cpu;
        }

        want_affine = !wake_wide(p) && cpumask_test_cpu(cpu, p->cpus_ptr);
    }

    rcu_read_lock();
    for_each_domain(cpu, tmp) {
        /*
         * If both 'cpu' and 'prev_cpu' are part of this domain,
         * cpu is a valid SD_WAKE_AFFINE target.
         */
        if (want_affine && (tmp->flags & SD_WAKE_AFFINE) &&
            cpumask_test_cpu(prev_cpu, sched_domain_span(tmp))) {
            if (cpu != prev_cpu)
                new_cpu = wake_affine(tmp, p, cpu, prev_cpu, sync);

            sd = NULL; /* Prefer wake_affine over balance flags */
            break;
        }

        /*
         * Usually only true for WF_EXEC and WF_FORK, as sched_domains
         * usually do not have SD_BALANCE_WAKE set. That means wakeup
         * will usually go to the fast path.
         */
        if (tmp->flags & sd_flag)
            sd = tmp;
        else if (!want_affine)
            break;
    }

    if (unlikely(sd)) {
        /* Slow path */
        new_cpu = sched_balance_find_dst_cpu(sd, p, cpu, prev_cpu, sd_flag);
    } else if (wake_flags & WF_TTWU) { /* XXX always ? */
        /* Fast path */
        new_cpu = select_idle_sibling(p, prev_cpu, new_cpu);
    }

    rcu_read_unlock();
}

```

```
    return new_cpu;  
}
```

task1/p4_select_idle_sibling.txt

kernel source: select_idle_sibling (the placement decision)

```

static int select_idle_sibling(struct task_struct *p, int prev, int target)
{
    bool has_idle_core = false;
    struct sched_domain *sd;
    unsigned long task_util, util_min, util_max;
    int i, recent_used_cpu, prev_aff = -1;

    /*
     * On asymmetric system, update task utilization because we will check
     * that the task fits with CPU's capacity.
     */
    if (sched_asym_cpucap_active()) {
        sync_entity_load_avg(&p->se);
        task_util = task_util_est(p);
        util_min = uclamp_eff_value(p, UCLAMP_MIN);
        util_max = uclamp_eff_value(p, UCLAMP_MAX);
    }

    /*
     * per-cpu select_rq_mask usage
     */
    lockdep_assert_irqs_disabled();

    if ((available_idle_cpu(target) || sched_idle_cpu(target)) &&
        asym_fits_cpu(task_util, util_min, util_max, target))
        return target;

    /*
     * If the previous CPU is cache affine and idle, don't be stupid:
     */
    if (prev != target && cpus_share_cache(prev, target) &&
        (available_idle_cpu(prev) || sched_idle_cpu(prev)) &&
        asym_fits_cpu(task_util, util_min, util_max, prev)) {

        if (!static_branch_unlikely(&sched_cluster_active) ||
            cpus_share_resources(prev, target))
            return prev;

        prev_aff = prev;
    }

    /*
     * Allow a per-cpu kthread to stack with the wakee if the
     * kworker thread and the tasks previous CPUs are the same.
     * The assumption is that the wakee queued work for the
     * per-cpu kthread that is now complete and the wakeup is
     * essentially a sync wakeup. An obvious example of this
     * pattern is IO completions.
     */
    if (is_per_cpu_kthread(current) &&
        in_task() &&
        prev == smp_processor_id() &&
        this_rq()->nr_running <= 1 &&
        asym_fits_cpu(task_util, util_min, util_max, prev)) {
        return prev;
    }

    /* Check a recently used CPU as a potential idle candidate: */
    recent_used_cpu = p->recent_used_cpu;
    p->recent_used_cpu = prev;
    if (recent_used_cpu != prev &&
        recent_used_cpu != target &&
        cpus_share_cache(recent_used_cpu, target) &&
        (available_idle_cpu(recent_used_cpu) || sched_idle_cpu(recent_used_cpu)) &&
        cpumask_test_cpu(recent_used_cpu, p->cpus_ptr) &&
        asym_fits_cpu(task_util, util_min, util_max, recent_used_cpu)) {

```

```

        if (!static_branch_unlikely(&sched_cluster_active) ||
            cpus_share_resources(recent_used_cpu, target))
            return recent_used_cpu;

    } else {
        recent_used_cpu = -1;
    }

    /*
     * For asymmetric CPU capacity systems, our domain of interest is
     * sd_asym_cpucapacity rather than sd_llc.
     */
    if (sched_asym_cpucap_active()) {
        sd = rcu_dereference_all(per_cpu(sd_asym_cpucapacity, target));
        /*
         * On an asymmetric CPU capacity system where an exclusive
         * cpuset defines a symmetric island (i.e. one unique
         * capacity_orig value through the cpuset), the key will be set
         * but the CPUs within that cpuset will not have a domain with
         * SD_ASYM_CPUCAPACITY. These should follow the usual symmetric
         * capacity path.
         */
        if (sd) {
            i = select_idle_capacity(p, sd, target);
            return ((unsigned)i < nr_cpumask_bits) ? i : target;
        }
    }

    sd = rcu_dereference_all(per_cpu(sd_llc, target));
    if (!sd)
        return target;

    if (sched_smt_active()) {
        has_idle_core = test_idle_cores(target);

        if (!has_idle_core && cpus_share_cache(prev, target)) {
            i = select_idle_smt(p, sd, prev);
            if ((unsigned int)i < nr_cpumask_bits)
                return i;
        }
    }

    i = select_idle_cpu(p, sd, has_idle_core, target);
    if ((unsigned)i < nr_cpumask_bits)
        return i;

    /*
     * For cluster machines which have lower sharing cache like L2 or
     * LLC Tag, we tend to find an idle CPU in the target's cluster
     * first. But prev_cpu or recent_used_cpu may also be a good candidate,
     * use them if possible when no idle CPU found in select_idle_cpu().
     */
    if ((unsigned int)prev_aff < nr_cpumask_bits)
        return prev_aff;
    if ((unsigned int)recent_used_cpu < nr_cpumask_bits)
        return recent_used_cpu;

    return target;
}

```

Task 2

task2/p10_reboot_confirmed.txt

the gap, with perf stat (dTLB collapse) and smaps (FilePmdMapped=64MiB for v2)

```
# Task 2 – the gap, and the counters that explain it (course-08, kernel 7.0.0-15-generic)
# COMMON="--file-num=1 --file-total-size=64M"
# RUN="sysbench fileio $COMMON --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run"

$ (cd ~/v1 && $RUN) | grep reads/s
reads/s: 1942670.32
$ (cd ~/v2 && $RUN) | grep reads/s
reads/s: 2124442.66 # v2 is +9.4%

$ (cd ~/v1 && sudo perf stat -e cycles,instructions,dTLB-load-misses,cache-misses,cache-
references $RUN)
reads/s: 1886428.41
13255912155 cycles
7538218104 instructions
8790240 dTLB-load-misses
130604123 cache-misses
634898144 cache-references

$ (cd ~/v2 && sudo perf stat -e cycles,instructions,dTLB-load-misses,cache-misses,cache-
references $RUN)
reads/s: 2082856.51
13263522195 cycles
8306823225 instructions
18488 dTLB-load-misses
126771643 cache-misses
676552167 cache-references

# dTLB-load-misses: v1 = 8,790,240 vs v2 = 18,488 (about 475x fewer for v2)
# cache-misses (LLC): 130.6M vs 126.8M (about equal) -> the win is the TLB, not the cache

$ (cd ~/v2 && $RUN) & sleep 2; grep -E 'FilePmdMapped|Rss' /proc/$(pgrep -n
sysbench)/smaps_rollup
Rss: 74872 kB
FilePmdMapped: 65536 kB # v2: the whole 64 MiB file is mapped with 2 MiB huge pages

$ (cd ~/v1 && $RUN) & sleep 2; grep -E 'FilePmdMapped|Rss' /proc/$(pgrep -n
sysbench)/smaps_rollup
Rss: 74972 kB
FilePmdMapped: 0 kB # v1: no huge mapping
```

task2/p5_folio_hist_v1.txt

folio-order histogram, v1


```
# 5a folio-order histogram - v1 (4K prepare)
# NOTE: kprobe:filemap_alloc_folio NOT probeable on this kernel (inlined).
#       Documented fallback: tracepoint:filemap:mm_filemap_add_to_page_cache, field 'order'
#       (verified via -lv).
#       Histogram bucket = folio order (0=4K page,1=8K,2=16K,...,9=2M PMD). File removed pre-
#       prepare so writes run.
# kernel: 7.0.0-15-generic   Sun Jun 14 11:42:00 UTC 2026
```

```
Attached 1 probe
sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)
```

```
1 files, 65536Kb each, 64Mb total
Creating files for the test...
Extra file open flags: (none)
Creating file test_file.0
67108864 bytes written in 0.05 seconds (1254.36 MiB/sec).
```

```
@order_v1:
[0]          3027 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[1]           0 | |
[2]         4096 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
```

task2/p5_folio_hist_v2.txt

folio-order histogram, v2

```
# 5a folio-order histogram - v2 (4M prepare)
# Same probe substitution as v1: tracepoint:filemap:mm_filemap_add_to_page_cache, field 'order'.
#       Histogram bucket = folio order (0=4K,1=8K,2=16K,...,9=2M PMD). File removed pre-prepare
#       so writes run.
# kernel: 7.0.0-15-generic   Sun Jun 14 11:42:10 UTC 2026
```

```
Attached 1 probe
sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)
```

```
1 files, 65536Kb each, 64Mb total
Creating files for the test...
Extra file open flags: (none)
Creating file test_file.0
67108864 bytes written in 0.04 seconds (1467.75 MiB/sec).
```

```
@order_v2:
[0]          2870 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[1]           0 | |
[2]           0 | |
[3]           0 | |
[4]           0 | |
[5]           0 | |
[6]           0 | |
[7]           0 | |
[8]           0 | |
[9]           32 | |
```

task2/p1b_filefrag.txt

filefrag (fragmentation is trivial)

```
/home/ubuntu/v1/test_file.0: 2 extents found
/home/ubuntu/v2/test_file.0: 1 extent found
```

task2/p2_perfststat_v1.txt

perf stat v1 (page-faults are all minor -> fully cached)

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:

Number of threads: 1

Initializing random number generator from current time

Extra file open flags: (none)

1 files, 64MiB each

64MiB total file size

Block size 4KiB

Number of IO requests: 0

Read/Write ratio for combined random IO test: 1.50

Periodic FSYNC enabled, calling fsync() each 100 requests.

Calling fsync() at the end of test, Enabled.

Using fast mmaped I/O mode

Doing random read test

Initializing worker threads...

Threads started!

File operations:

reads/s: 1882830.28

writes/s: 0.00

fsyncs/s: 0.00

Throughput:

read, MiB/s: 7354.81

written, MiB/s: 0.00

General statistics:

total time: 5.0002s

total number of events: 9417768

Latency (ms):

min: 0.00

avg: 0.00

max: 0.93

95th percentile: 0.00

sum: 3221.33

Threads fairness:

events (avg/stddev): 9417768.0000/0.00

execution time (avg/stddev): 3.2213/0.00

Performance counter stats for 'sysbench fileio --file-num=1 --file-total-size=64M --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run':

1813	page-faults
1813	minor-faults
8744015	dTLB-load-misses
13255640893	cycles
7529345671	instructions

5.008987211 seconds time elapsed

4.992305000 seconds user

0.008995000 seconds sys

task2/p5c_force_thp.txt

independent check: 2 MiB (MAP_HUGETLB) vs 4 KiB pages, same read pattern

=== 256MB mode=0 ===

mode=0 MB=256 AnonHugePages_kB=0

mode=0 MB=256 reads/s=62759989 sum=200000000

Performance counter stats for '/tmp/thp_rndrd2 256 0':

194681566	dTLB-load-misses
8881399109	cycles
3731522806	instructions

3.359090688 seconds time elapsed

3.192268000 seconds user

0.164014000 seconds sys

=== 256MB mode=1 ===

MADV_COLLAPSE failed: Invalid argument

mode=1 MB=256 AnonHugePages_kB=0

mode=1 MB=256 reads/s=62624913 sum=200000000

Performance counter stats for '/tmp/thp_rndrd2 256 1':

194680010	dTLB-load-misses
8903528253	cycles
3725908083	instructions

3.365086766 seconds time elapsed

3.205633000 seconds user

0.158970000 seconds sys

=== 1024MB mode=0 ===

mode=0 MB=1024 AnonHugePages_kB=0

mode=0 MB=1024 reads/s=56452905 sum=200000000

Performance counter stats for '/tmp/thp_rndrd2 1024 0':

198810736	dTLB-load-misses
11112291706	cycles
4656052798	instructions

4.230569558 seconds time elapsed

3.592510000 seconds user

0.636054000 seconds sys

=== 1024MB mode=1 ===

MADV_COLLAPSE failed: Invalid argument

mode=1 MB=1024 AnonHugePages_kB=0

mode=1 MB=1024 reads/s=56409139 sum=200000000

Performance counter stats for '/tmp/thp_rndrd2 1024 1':

198815712	dTLB-load-misses
11153220204	cycles
4654098855	instructions

4.242477699 seconds time elapsed

3.609043000 seconds user

0.632823000 seconds sys

```
HugePages_Total:    600
HugePages_Free:    600
=== 256MB mode=2 (MAP_HUGETLB) ===
mode=2 MB=256 AnonHugePages_kB=0 (hugetlb: see HugePages_Free)
mode=2 MB=256 reads/s=70738072 sum=200000000
```

Performance counter stats for '/tmp/thp_rndrd2 256 2':

2753	dTLB-load-misses
7657971203	cycles
3429336113	instructions

2.883778716 seconds time elapsed

2.859685000 seconds user
0.020998000 seconds sys

```
=== 1024MB mode=2 (MAP_HUGETLB) ===
mode=2 MB=1024 AnonHugePages_kB=0 (hugetlb: see HugePages_Free)
mode=2 MB=1024 reads/s=67850882 sum=200000000
```

Performance counter stats for '/tmp/thp_rndrd2 1024 2':

13578	dTLB-load-misses
8412641647	cycles
3455270546	instructions

3.159761606 seconds time elapsed

3.075510000 seconds user
0.081935000 seconds sys

task2/p4_page_cache_ra_order.txt

kernel source: page_cache_ra_order (folio order capped by I/O size)

```

void page_cache_ra_order(struct readahead_control *ractl,
                        struct file_ra_state *ra)
{
    struct address_space *mapping = ractl->mapping;
    pgoff_t start = readahead_index(ractl);
    pgoff_t index = start;
    unsigned int min_order = mapping_min_folio_order(mapping);
    pgoff_t limit = (i_size_read(mapping->host) - 1) >> PAGE_SHIFT;
    pgoff_t mark = index + ra->size - ra->async_size;
    unsigned int nofs;
    int err = 0;
    gfp_t gfp = readahead_gfp_mask(mapping);
    unsigned int new_order = ra->order;

    trace_page_cache_ra_order(mapping->host, start, ra);
    if (!mapping_large_folio_support(mapping)) {
        ra->order = 0;
        goto fallback;
    }

    limit = min(limit, index + ra->size - 1);

    new_order = min(mapping_max_folio_order(mapping), new_order);
    new_order = min_t(unsigned int, new_order, ilog2(ra->size));
    new_order = max(new_order, min_order);

    ra->order = new_order;

    /* See comment in page_cache_ra_unbounded() */
    nofs = memalloc_nofs_save();
    filemap_invalidate_lock_shared(mapping);
    /*
     * If the new_order is greater than min_order and index is
     * already aligned to new_order, then this will be noop as index
     * aligned to new_order should also be aligned to min_order.
     */
    ractl->_index = mapping_align_index(mapping, index);
    index = readahead_index(ractl);

    while (index <= limit) {
        unsigned int order = new_order;

        /* Align with smaller pages if needed */
        if (index & ((1UL << order) - 1))
            order = __ffs(index);
        /* Don't allocate pages past EOF */
        while (order > min_order && index + (1UL << order) - 1 > limit)
            order--;
        err = ra_alloc_folio(ractl, index, mark, order, gfp);
        if (err)
            break;
        index += 1UL << order;
    }

    read_pages(ractl);
    filemap_invalidate_unlock_shared(mapping);
    memalloc_nofs_restore(nofs);

    /*
     * If there were already pages in the page cache, then we may have
     * left some gaps. Let the regular readahead code take care of this
     * situation below.
     */
    if (!err)
        return;
    fallback:

```

```
/*
 * ->readahead() may have updated readahead window size so we have to
 * check there's still something to read.
 */
if (ra->size > index - start)
    do_page_cache_ra(ractl, ra->size - (index - start),
        ra->async_size);
}
```

task2/p4_do_set_pmd.txt

kernel source: do_set_pmd (installs the 2 MiB huge-page mapping)

```

vm_fault_t do_set_pmd(struct vm_fault *vmf, struct folio *folio, struct page *page)
{
    struct vm_area_struct *vma = vmf->vma;
    bool write = vmf->flags & FAULT_FLAG_WRITE;
    unsigned long haddr = vmf->address & HPAGE_PMD_MASK;
    pmd_t entry;
    vm_fault_t ret = VM_FAULT_FALLBACK;

    /*
     * It is too late to allocate a small folio, we already have a large
     * folio in the pagecache: especially s390 KVM cannot tolerate any
     * PMD mappings, but PTE-mapped THP are fine. So let's simply refuse any
     * PMD mappings if THPs are disabled. As we already have a THP,
     * behave as if we are forcing a collapse.
     */
    if (thp_disabled_by_hw() || vma_thp_disabled(vma, vma->vm_flags,
        /* forced_collapse=*/ true))
        return ret;

    if (!thp_vma_suitable_order(vma, haddr, PMD_ORDER))
        return ret;

    if (folio_order(folio) != HPAGE_PMD_ORDER)
        return ret;
    page = &folio->page;

    /*
     * Just backoff if any subpage of a THP is corrupted otherwise
     * the corrupted page may mapped by PMD silently to escape the
     * check. This kind of THP just can be PTE mapped. Access to
     * the corrupted subpage should trigger SIGBUS as expected.
     */
    if (unlikely(folio_test_has_hwpoisoned(folio)))
        return ret;

    /*
     * Archs like ppc64 need additional space to store information
     * related to pte entry. Use the preallocated table for that.
     */
    if (arch_needs_pgtable_deposit() && !vmf->prealloc_pte) {
        vmf->prealloc_pte = pte_alloc_one(vma->vm_mm);
        if (!vmf->prealloc_pte)
            return VM_FAULT_OOM;
    }

    vmf->ptl = pmd_lock(vma->vm_mm, vmf->pmd);
    if (unlikely(!pmd_none(*vmf->pmd)))
        goto out;

    flush_icache_pages(vma, page, HPAGE_PMD_NR);

    entry = folio_mk_pmd(folio, vma->vm_page_prot);
    if (write)
        entry = maybe_pmd_mkdirty(pmd_mkdirty(entry), vma);

    add_mm_counter(vma->vm_mm, mm_counter_file(folio), HPAGE_PMD_NR);
    folio_add_file_rmap_pmd(folio, page, vma);

    /*
     * deposit and withdraw with pmd lock held
     */
    if (arch_needs_pgtable_deposit())
        deposit_prealloc_pte(vmf);

    set_pmd_at(vma->vm_mm, haddr, vmf->pmd, entry);
}

```



```
    update_mmu_cache_pmd(vma, haddr, vmf->pmd);

    /* fault is handled */
    ret = 0;
    count_vm_event(THP_FILE_MAPPED);
out:
    spin_unlock(vmf->ptl);
    return ret;
}
```